

反向传播

1 单层神经网络

单层神经网络可以看作是一个简单的线性变换后接一个激活函数：

$$L(x) = f(W \cdot x + b)$$

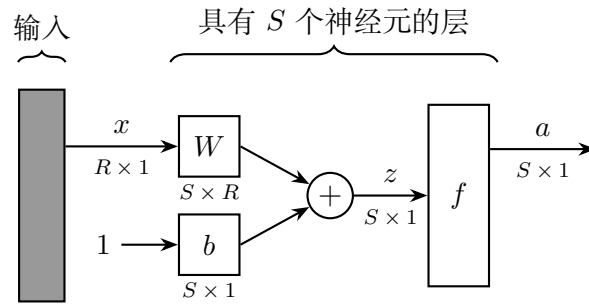


图 1.1: 单层神经网络

2 多层感知机 (MLP)

多层感知机就是多个单层网络的叠加（复合）：

$$MLP(x) = L^L \circ L^{L-1} \circ \dots \circ L^1(x)$$

其中每一层 $L^l(\cdot) = f^l(W^l \cdot + b^l)$ 。将每层的“线性部分”和“非线性部分”分开看：

$$\underbrace{z^l = W^l a^{l-1} + b^l}_{\text{线性：参数都在这里}} \quad \underbrace{a^l = f(z^l)}_{\text{非线性：无参数}}$$

3 反向传播 (BP) 算法推导

反向传播旨在迭代地计算梯度。

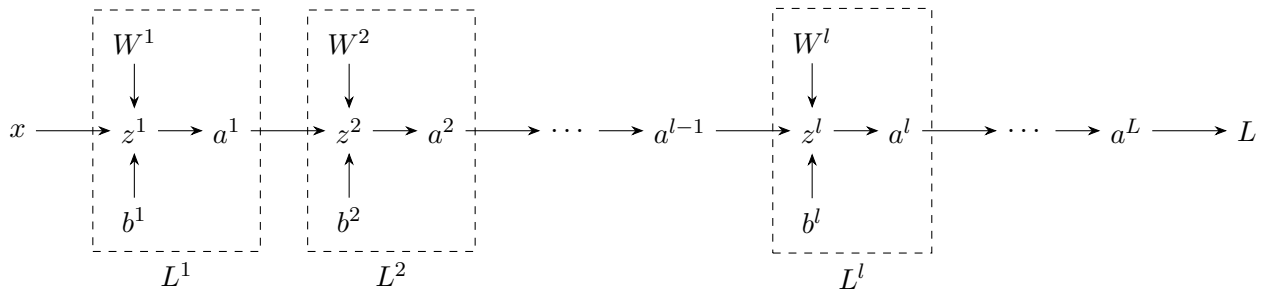


图 3.1: MLP 中的变量依赖关系

我们的目标是计算 $\frac{\partial L}{\partial W^l}$ 与 $\frac{\partial L}{\partial b^l}$ ，对每一层 l 。但困难是 W^l 离 L 隔着许多层，如果要直接对它求偏导，需要把链式法则一路推到底。逐个层地展开，最终会得到一个非常长的链式乘积，计算量大且容易出错。所以我们需要利用一些中间变量来分解这个长链式，迭代地计算梯度。

3.1 参数梯度的分解

注意到一个自然的**简化**：所有从 W^l 和 b^l 出发去往 L 的路径，总是可以分解为先到达 z^l ，再从 z^l 到 L 。所以只要拿到误差项

$$\delta^l = \frac{\partial L}{\partial z^l}$$

剩下的就是 z^l 怎么显式依赖 W^l 和 b^l ，那一步是平凡的（线性）。这就把原来的“长链式”切成两段，于是计算过程重新组织为：先算每一层的 δ^l ，再用它合成参数梯度。

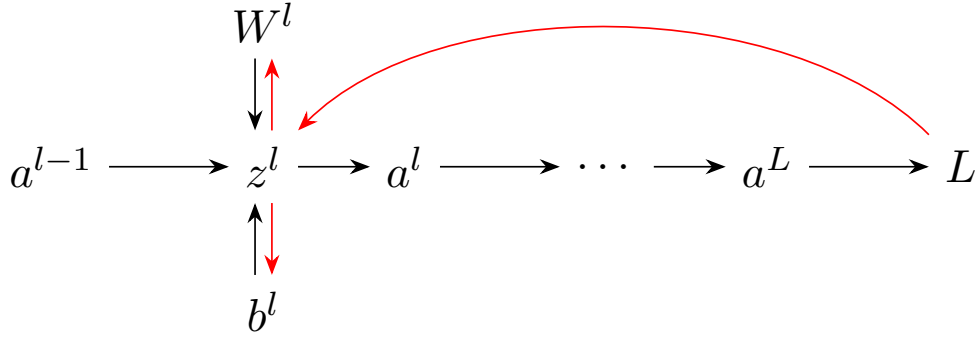


图 3.2: 参数梯度分解

接下来，我们使用**爱因斯坦求和约定**，假设已经得到第 l 层的误差项 δ^l ，计算该层参数梯度具体的分量：

$$\begin{aligned} \frac{\partial L}{\partial W_{ij}^l} &= \frac{\partial L}{\partial z_i^l} \frac{\partial z_i^l}{\partial W_{ij}^l} = \delta_i^l a_j^{l-1} \\ \frac{\partial L}{\partial b_i^l} &= \frac{\partial L}{\partial z_i^l} \frac{\partial z_i^l}{\partial b_i^l} = \delta_i^l \end{aligned}$$

3.2 误差项的递推公式 ($\delta^l \rightarrow \delta^{l-1}$)

想要从后往前地计算梯度，我们还需要误差项的递推公式，这是 BP 算法的核心。

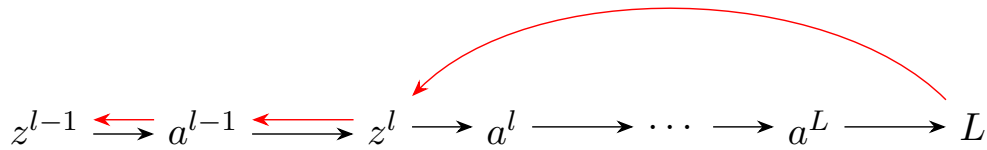


图 3.3: 误差项递推分解

$$\delta_i^{l-1} = \frac{\partial L}{\partial z_i^{l-1}} = \frac{\partial L}{\partial z_j^l} \frac{\partial z_j^l}{\partial a_k^{l-1}} \frac{\partial a_k^{l-1}}{\partial z_i^{l-1}} = \delta_j^l W_{jk}^l \frac{\partial a_k^{l-1}}{\partial z_i^{l-1}}$$

- **分量独立情况**（若 $a_k = f(z_k)$ ）：每个激活值仅取决于对应的净输入。

$$\delta_i^{l-1} = \delta_j^l W_{ji}^l \odot f'(z_i^{l-1})$$

- 分量相关情况 (如 Softmax):

$$\delta_i^{l-1} = \delta_j^l W_{jk}^l F_{ki}^{l-1}$$

其中 F_{ki}^{l-1} 是激活函数的雅可比矩阵。此时 j, k 为哑变量。

3.3 迭代的起点: δ^L

最后, 我们给这个递推公式一个“首项”。

$$z^L \xrightarrow{\quad} a^L \xrightarrow{\quad} L$$

图 3.4: δ^L 的计算

$$\delta_i^L = \frac{\partial L}{\partial a_j^L} \frac{\partial a_j^L}{\partial z_i^L}$$

- 分量不独立:

$$\delta_i^L = \frac{\partial L}{\partial a_j^L} F_{ji}^L$$

- 分量独立:

$$\delta_i^L = \frac{\partial L}{\partial a_i^L} \odot f'(z_i^L)$$

4 对照总结表

物理量	分量形式 (Einstein Convention)	矩阵形式
输出层误差 (独立)	$\delta_i^L = \frac{\partial L}{\partial a_i^L} \odot f'(z_i^L)$	$\delta^L = \frac{\partial L}{\partial a^L} \odot f'(z^L)$
输出层误差 (通用)	$\delta_i^L = \frac{\partial L}{\partial a_j^L} F_{ji}^L$	$\delta^L = (F^L)^T \frac{\partial L}{\partial a^L}$
权重梯度	$\frac{\partial L}{\partial W_{ij}^l} = \delta_i^l a_j^{l-1}$	$\frac{\partial L}{\partial W^l} = \delta^l (a^{l-1})^T$
偏置梯度	$\frac{\partial L}{\partial b_i^l} = \delta_i^l$	$\frac{\partial L}{\partial b^l} = \delta^l$
误差递推 (独立)	$\delta_i^{l-1} = \delta_j^l W_{ji}^l f'(z_i^{l-1})$	$\delta^{l-1} = (W^l)^T \delta^l \odot f'(z^{l-1})$
误差递推 (通用)	$\delta_i^{l-1} = \delta_j^l W_{jk}^l F_{ki}^{l-1}$	$\delta^{l-1} = (F^{l-1})^T (W^l)^T \delta^l$

表 4.1: 分量形式与矩阵形式对照

分量形式的好处就是转置位置一目了然 (看哑指标在哪一位), 且能无痛推广到张量 (卷积、batch 维度), 不必纠结“先转置还是先 reshape”。

5 总结

MLP 的反向传播算法的流程可以总结为:

1. 从输出层开始，计算 δ^L 。
2. 使用 δ^l 计算每层的参数梯度 $\frac{\partial L}{\partial W^l}$ 和 $\frac{\partial L}{\partial b^l}$ 。
3. 迭代地使用递推公式计算 δ^{l-1} 。
4. 重复步骤 2 和 3，直到计算到输入层。

6 引申：从反向传播到自动微分

我们并不满足只能在 MLP 上进行反向传播，接下来讨论如何在一般结构的网络上进行参数梯度的计算。

6.1 我们到底遇到了什么困难？

情形 1:

$$y = \theta_1^2 + \theta_2^2 + \cdots + \theta_n^2$$

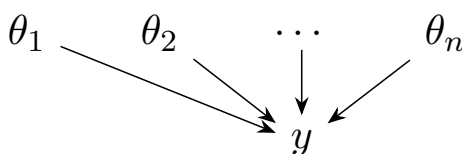


图 6.1: 情形 1 的计算图

此时

$$\frac{\partial y}{\partial \theta_i} = 2\theta_i$$

每一个分量可以对 y 偏导一次直接得到，每次求导的计算是不可避免的。

情形 2:

$$\begin{aligned} y &= \theta_1^2 + \varphi_1 \\ \theta_1 &= \theta_2^2 + \varphi_2 \\ \theta_2 &= \theta_3^2 + \varphi_3 \\ &\dots \\ \theta_{n-1} &= \theta_n^2 \end{aligned}$$

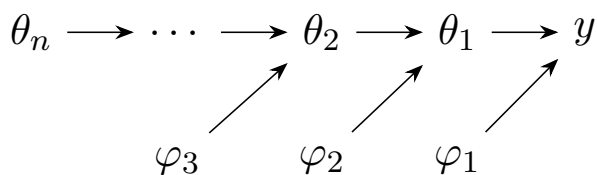


图 6.2: 情形 2 的计算图

通过链式法则计算：

$$\begin{aligned}\frac{\partial y}{\partial \varphi_1} &= 1 \\ \frac{\partial y}{\partial \varphi_2} &= \frac{\partial y}{\partial \theta_1} \cdot \frac{\partial \theta_1}{\partial \varphi_2} = 2\theta_1 \cdot 1 \\ &\vdots \\ \frac{\partial y}{\partial \theta_n} &= \frac{\partial y}{\partial \theta_1} \dots \frac{\partial \theta_{n-1}}{\partial \theta_n} = 2\theta_1 \cdot 2\theta_2 \dots 2\theta_n\end{aligned}$$

可以发现，在计算过程中出现了很多重复项，导致计算量巨大。

6.2 两者计算开销有本质区别！

假设：对 $y = f(x)$ ，计算 $\frac{\partial y}{\partial x}$ 需要 1 个单位时间。

- **情形 1：** 计算 $\frac{\partial y}{\partial \theta}$ 的所有分量需要 n 个单位时间，平均每个分量 1 单位时间。
- **情形 2：** 计算 $\frac{\partial y}{\partial \theta_i}$ 的第 i 个分量需要使用链式法则，计算 i 次偏导数。计算 n 个偏导数总共需要 $\frac{n(n+1)}{2}$ 单位时间，平均每个变量需 $\frac{n+1}{2}$ 单位时间。时间线性增长远大于常数。

在普通的梯度计算中，依赖链的长度越长，重复计算的次数就越多，导致计算效率急剧下降。但是现代神经网络的计算图非常复杂，依赖链无可避免的非常长，如果不加以优化，计算梯度的效率将会非常低下。

如何避免？ 谜底就在谜面上：用笔记本记下已经计算好的中间量，下次需要时直接取用即可（类似斐波那契数列的递归优化）。

6.3 反向传播的推导

反向传播算法的核心思想就是利用动态规划的思想，迭代地计算每个节点的梯度，并将其存储起来，以避免重复计算。如果要计算某个节点 v 的梯度 $\frac{\partial L}{\partial v}$ ，我们先计算它的所有直接后继节点 v' 的梯度 $\frac{\partial L}{\partial v'}$ ，然后利用链式法则将它们组合起来：

$$\frac{\partial L}{\partial v} = \sum_{v' \in \text{succ}(v)} \frac{\partial L}{\partial v'} \cdot \frac{\partial v'}{\partial v}$$

我们可以用计算图直观地理解这个过程：计算图中的任意节点 v 代表梯度 $\frac{\partial L}{\partial v}$ ，每个边代表一个局部梯度 $\frac{\partial v'}{\partial v}$ 。通过从输出层 L 开始，沿着计算图逆向传播，每经过一条边，我们就将当前节点的梯度“乘”以该边的局部梯度 $\frac{\partial v'}{\partial v}$ ，并将结果累加到前驱节点的梯度上。该前驱节点的梯度等于所有后继分支路径上所得梯度之和。

计算某个节点的梯度之前，必须先计算它的所有后继节点的梯度，因此反向传播的计算顺序是按照计算图的逆拓扑排序进行的。

6.4 与正向计算的联系

在正向计算中，对于某个节点 v ，我们要先计算它的所有前驱节点的值，然后才能计算 v 的值。节点的顺序对应计算图的拓扑排序。在反向传播中，对于某个节点 v ，我们要先计算它的所有后继

节点的梯度，然后才能计算 v 的梯度。节点的计算顺序对应计算图的逆拓扑排序。由于每个拓扑排序都有一个对应的逆拓扑排序，因此可以将正向计算的计算过程反向遍历一次，就可以得到反向传播的计算顺序。

6.5 自动微分的过程

结合前向计算和反向传播，我们可以构建出自动微分程序的整体流程。

1. **前向计算**：按照计算图的拓扑排序，从输入节点开始，逐层计算每个节点的值，并将其存储起来。
2. **反向传播**：按照计算图的逆拓扑排序，从输出节点 L 开始，逐层计算每个节点的梯度，并将其存储起来。
3. **参数更新**：使用计算得到的梯度来更新模型参数，例如通过梯度下降算法。

在这个过程中，前向计算和反向传播是紧密结合的，前向计算为反向传播提供了必要的中间值，而反向传播则利用这些中间值高效地计算梯度，从而实现了自动微分的功能。

在构建一个自动微分程序时，我们需要设计一个数据结构来表示计算图中的节点和边，并实现前向计算和反向传播的算法。通常，节点会包含一个值和一个梯度，以及指向其前驱节点和后继节点的指针。边会定义如何用前驱节点的值计算后继节点，同时定义如何将后继节点的梯度“乘”以局部梯度来更新前驱节点的梯度。